

OpenCode Build Prompts — Vantage Robotics Simulation Suite

Paste these into OpenCode (DeepSeek) in order. This version has been expanded with full context, numbered step-by-step instructions, acceptance criteria, and a verification checklist per prompt — DeepSeek performs better with explicit, self-contained instructions than short prose, so don't trim these down when pasting.

How to use this file

- **Mode:** Plan mode (Tab to toggle) for Prompts 0, 1, 6, 7, and 14 — architecture-defining steps where you want a proposal before files get touched. Everything else: Build mode.
 - **Tool split:** Prompts 0–12 and 14–17 are structural/logic — build these with OpenCode/DeepSeek. Prompt 13 (visual design) is the one for **Antigravity** — a full design brief is included so you're not starting from a blank prompt there either.
 - **Commit after every prompt**, using the exact commit message given, even for small steps — this is your `/undo` safety net.
 - Run `/check-secrets` before every commit touching `.env` or config.
 - Copy `AGENTS.md`, `DEPLOYMENT.md`, `PROJECT_OVERVIEW.md`, `6_dof_arm.urdf`, and `key_config.json` into the repo root **before Prompt 0**. DeepSeek needs these physically present in the repo to read them — don't just paste snippets into chat.
 - After each prompt: run the step's verification checklist before committing. If a checklist item fails, tell DeepSeek what failed and ask it to fix that specific item — don't move to the next prompt with a known-broken checklist item.
-

Phase mapping — quick reference

Prompt	Title	PDF Phase	Rubric weight
0	Context primer	Pre-Phase	—
1	Architecture plan	Pre-Phase	—
2	Scaffold the app	Pre-Phase	—
3	URDF viewer	Phase 1	Visualization & Dashboard, 15%
4	Key panel render	Phase 1	Visualization & Dashboard, 15%
5	Telemetry dashboard	Phase 1	Visualization & Dashboard, 15%
6	IK solver	Phase 2	Inverse Kinematics, 15%
7	MotionController	Phase 2 (foundation for all)	Inverse Kinematics 15% + Architecture 15%
8	Joystick control	Phase 2	Manual Control, 10%
9	Keyboard control	Phase 2	Manual Control, 10%
10	Autonomous PIN entry	Phase 4	Autonomous PIN Entry, 20%
11	Voice control (deterministic)	Phase 3	Voice Control, 15%
12	Unify dashboard	Integration (Phases 1-4)	Architecture 15% + Polish 5%
13	Visual design (Antigravity)	— (rubric only)	Overall Polish & Presentation, 5%
14	Agentic voice bonus	Phase 3B	+10% bonus
15	Deploy-readiness	—	Deliverables (deployed URL, bonus)
16	Judge documentation	— (rubric only)	Architecture & Concept Explanation, 15%
17	Electrical schematic	Phase 5	Electrical Schematic, 5%

Evaluation notes carried over from the prior review

- Prompt 10 (Phase 4) is intentionally built before Prompt 11 (Phase 3)** — PIN entry is worth 20%, voice is worth 15%; higher-value work goes first. Tell your whole team this is deliberate.
- Demo order ≠ build order.** Demo/pitch should follow the PDF's stated story: visualize → manual → voice → autonomous PIN entry, regardless of build sequence.
- IK convergence tolerance must be tighter than the ±5mm rubric tolerance** for PIN entry — baked into Prompt 6 below.
- Phase 3B must never replace the required Phase 3 deterministic baseline** — baked into Prompt 14 below as an explicit guard.

Prompt 0 — Full context primer (Plan mode, no commit)

Paste this first, before Prompt 1. Its only job is to load maximum context into DeepSeek's working memory before any planning or coding begins.

Before we do anything else, read every one of these files in this repo fully: AGENTS.md, DEPLOYMENT.md, PROJECT_OVERVIEW.md, 6_dof_arm.urdf, key_config.json. Do not summarize back at length — just confirm you've read them, then answer these specific questions to prove you've actually absorbed the details, not skimmed:

1. How many actuated revolute joints does the URDF define, and what is each one's axis (X/Y/Z) and joint limit in degrees?
2. What is the exact name of the true end-effector frame, and how far (in meters) is it offset from the last actuated joint (stylus_pitch)?
3. What are the (x, y, z) coordinates of key 3 and key 4 from key_config.json, and what does "approach_axis: -z" mean for how the arm should move to press a key?
4. What is the exact TypeScript union type for MotionCommand as specified in PROJECT_OVERVIEW.md, and which single component is allowed to write to joint state?
5. What are the 8 scored rubric criteria and their weights, and which one is worth the most?

Answer all five before we proceed to Prompt 1. If anything in the source files is ambiguous or contradictory, say so now rather than silently picking an interpretation.

No commit. Read the answers back and correct DeepSeek if anything is wrong before continuing — this is the cheapest point in the whole day to catch a misunderstanding.

Prompt 1 — Architecture plan (Plan mode, no commit yet) — *Pre-Phase (spans all Phases)*

Now propose a complete file/folder structure for a Next.js App Router project implementing the full system described in PROJECT_OVERVIEW.md. Be specific — list actual paths, not just categories. For each of the following, tell me the exact file path you'd create and one line on why:

1. The URDF/Three.js viewer component
2. The shared joint-state store (how components read/write joint angles — note: only the MotionController should WRITE to this)
3. The IK solver module (must be framework-agnostic, no React/Three.js imports inside it)
4. The MotionController module
5. The MotionCommand type definitions (one shared source of truth)
6. Each control adapter: joystick, keyboard, voice, PIN sequencer, agentic bonus route
7. The key panel config loader/validator
8. The dashboard layout / activity log component
9. Where tests live, and what testing approach you'd use for the IK solver specifically (this is the one piece of logic worth unit testing given it's pure math)

Do not write code yet. List the structure as a file tree, then flag anything in PROJECT_OVERVIEW.md you think is ambiguous or where you'd make a judgment call, so I can confirm before you start building.

No commit — planning step. Review the tree as a team; adjust anything that doesn't match how you want to divide work between teammates before Prompt 2.

Checklist before moving on:

- ☐ Every one of the 9 numbered items above has a concrete file path
- ☐ The joint-state store's write-access rule (MotionController only) is explicitly stated in the plan
- ☐ The IK solver is confirmed framework-agnostic (no React/Three.js imports)
- ☐ Any ambiguities DeepSeek flagged have been discussed and resolved by the team

Prompt 2 — Scaffold the app (Build mode) — *Pre-Phase (setup, before Phase 1)*

Scaffold the Next.js 15 App Router project using the folder structure we just agreed on in Prompt 1. Specifically:

1. Run/generate package.json with: next, react, react-dom, three, @types/three, zod as dependencies; typescript, tailwindcss, postcss, autoprefixer, @types/node, @types/react, @types/react-dom as devDependencies. Do NOT add @supabase/supabase-js or any database client — this project has zero backend persistence, all state is client-side simulation state that resets on page reload, which is fine for a demo.
2. Create tsconfig.json, next.config.js, tailwind.config.js, postcss.config.js with standard Next.js 15 + Tailwind defaults.
3. Create .env.example containing ONLY: GEMINI_API_KEY= and GROQ_API_KEY= (both commented out with # for now — they're only needed for Prompt 14, the optional bonus). Leave a comment explaining this file intentionally has nothing required for the core pipeline.
4. Create .gitignore covering node_modules/, .next/, .env*.local, *.log.
5. Create the empty folder structure from Prompt 1's plan, with a one-line placeholder README.md or index file in each so empty folders aren't lost by git.
6. Create app/layout.tsx (minimal, Tailwind globals imported) and a placeholder app/page.tsx that just renders "Vantage Robotics Simulation Suite — scaffold OK".
7. Create app/api/health/route.ts returning { status: "ok" }.
8. Run npm install and confirm `npm run dev` starts with no errors.

Commit: chore: scaffold Next.js app with agreed folder structure

Checklist before moving on:

- ☐ npm run dev starts with zero errors or warnings in the terminal
- ☐ Visiting localhost shows the placeholder text
- ☐ /api/health returns {"status":"ok"}
- ☐ .env.example has no real keys, only empty/commented placeholders
- ☐ git status shows no node_modules or .next staged

Prompt 3 — Load and render the URDF (Build mode) — Problem Statement Phase 1 ("See the Arm")

Implement the 3D viewer. Follow these steps exactly:

1. Install urdf-loader (the gkjohnson package) and confirm it's added to package.json.
2. Create the Three.js scene component at the path we agreed in Prompt 1. It must include: a PerspectiveCamera, OrbitControls (so judges/mentors can rotate the view during demo), a ground grid (Three.js GridHelper) for scale reference, an ambient light plus one directional light with shadows enabled.
3. Load 6_dof_arm.urdf using URDFLoader. Confirm the loaded robot has exactly 7 actuated joints: joint_1, joint_2, joint_3, joint_4, joint_5, joint_6, stylus_pitch — plus the fixed stylus_tip_frame. Log this list to the console once on load so we can visually confirm in devtools.
4. Create the shared joint-state store (per the Prompt 1 plan) that holds the current angle of all 7 joints plus a method to read the computed world position of the stylus_tip frame. This store must be readable by any component, but do NOT wire up any write access yet — that's Prompt 7's job (the MotionController). For now, only this viewer component may set initial/default angles (e.g. all zeros) on load.
5. On every animation frame, sync the Three.js robot's joint angles FROM the store (read-only for this component after initial load).
6. Add a loading state (simple text or spinner) shown while the URDF parses, and an error state if the URDF fails to load.

Do not implement any interactivity yet (no dragging, no controls) — this prompt is purely: load, render, expose state for later use.

Commit: feat: load and render URDF arm in Three.js viewer with shared joint state access

SS

Checklist before moving on:

- ☐ All 7 joints + stylus_tip_frame appear in the console log on load
- ☐ Arm renders visually correct (compare proportions to the URDF comments: base, then 3 long 0.25m links, then 2 short 0.15m links, then the stylus)
- ☐ OrbitControls works (can rotate/zoom the camera with mouse)
- ☐ Loading state shows briefly, error state can be triggered by temporarily renaming the URDF path to confirm it works
- ☐ No console errors/warnings related to Three.js or the loader

Prompt 4 — Render the key panel (Build mode) — *Phase 1, task 3*

Render the 6-key test panel. Steps:

1. Create `keyConfig.ts` (or the path agreed in Prompt 1) that imports `key_config.json`, validates its shape with `zod` (`frame: string`, `units: string`, `approach_axis: string`, `keys: record of 6 entries each with x/y/z numbers`), and exports a typed, validated constant. No other file should import the raw JSON directly — always import from this module.
2. In the same Three.js scene as Prompt 3, add six small colored boxes (`BoxGeometry`, roughly 20-30mm per side) positioned at each key's (`x`, `y`, `z`) from the validated config, in the `base_link` frame — same root frame as the URDF, so no coordinate transform is needed, just place them directly.
3. Color-code the two rows differently (e.g. keys 1-2-3 one color, keys 4-5-6 another) purely so it's visually easy to tell them apart during demo — this has no functional meaning, just clarity.
4. Add a text label for each key showing its number (1-6), using either a Three.js `Sprite` with a canvas texture, or an HTML overlay positioned via the camera projection (your choice — pick whichever is less code). This satisfies the PDF's optional bonus: "digit labels on the panel... attempt as a bonus."
5. Confirm visually that the panel sits in front of and below the arm's base at a sensible reachable distance (per `PROJECT_OVERVIEW.md` Section 3.3, this should be well within the ~1.5m max reach).

Commit: `feat: render 6-key test panel from key_config.json`

Checklist before moving on:

- ☐ All 6 keys render at visually distinct, correctly separated positions (3 in a row, 2 rows)
- ☐ Key numbers 1-6 are legible in the 3D view
- ☐ Panel position looks reachable relative to the arm base (not absurdly far or intersecting the base)
- ☐ `keyConfig.ts` is the only place that imports the raw JSON (grep the codebase to confirm)

Prompt 5 — Live joint/telemetry dashboard (Build mode) — Phase 1, task 2

Build a live telemetry panel. Requirements:

1. Create a dashboard component (sidebar or bottom panel — your call, but leave room in the layout for controls to be added in later prompts) that reads from the SAME joint-state store created in Prompt 3. Do not create a second copy of joint-angle state anywhere.
2. Display all 7 joint angles, converted from radians to degrees, updated live (every animation frame or on every store change — whichever is simpler given your state management choice).
3. Compute and display the current (x, y, z) world position of the `stylus_tip` frame (forward kinematics from the current joint angles) — this is the position judges will watch to confirm the arm is actually at the key coordinates during PIN entry later.
4. Format numbers sensibly: angles to 1 decimal place, positions to 3 decimal places (millimeter precision, since the $\pm 5\text{mm}$ tolerance matters later).
5. Add a small visual indicator (color or icon) if any joint is within 5 degrees of its limit — this is a nice-to-have that helps operators trust the simulation, matching the "engineers need to trust what they see" framing from the problem's story section.

Commit: `feat: add live joint angle and end-effector telemetry dashboard`

Checklist before moving on:

- ☐ Telemetry numbers update in real time as the store changes (test by manually setting a joint angle in devtools/console if no controls exist yet)
- ☐ End-effector position matches expectations at the zero/default pose (compute by hand or sanity-check against the URDF's link lengths)
- ☐ Joint-limit warning indicator triggers correctly near a limit

Prompt 6 — IK solver (Plan mode first, then Build) — *Problem Statement Phase 2 ("Move the Arm"), task 1*

[Plan mode] Propose the design for a numerical inverse kinematics solver (damped least squares Jacobian) that takes a target (x, y, z) for the stylus_tip frame and returns joint angles for all 7 joints. Per PROJECT_OVERVIEW.md Section 3.6, this should be a position-only solver (3 target rows), treating the redundant stylus_pitch joint naturally via the minimum-norm property of DLS — don't lock any joint. State your damping factor choice, iteration limit, and convergence tolerance before writing code. The convergence tolerance MUST be tighter than the $\pm 5\text{mm}$ success tolerance defined in the problem statement's Phase 4 (autonomous PIN entry) — e.g. converge to within 1-2mm, not 5mm, so downstream position error from animation/interpolation doesn't push a technically-converged solution back outside the $\pm 5\text{mm}$ window that actually counts for scoring. Explain how you'll compute the Jacobian numerically (finite differences) versus analytically, and why.

Once reviewed, switch to Build mode:

Implement the IK solver exactly as planned. Requirements:

1. Pure function/module: takes (currentAngles: number[], jointAxes, jointLimits, targetPosition: [x,y,z]) and returns { angles: number[], converged: boolean, iterations: number, finalError: number }. No React, no Three.js scene object dependency — it should work given only the kinematic chain description (axes, link offsets, limits), which you can extract once from the loaded URDF and pass in as plain data.
2. Respect joint limits DURING iteration (clamp each iteration step, not just at the end) so it never proposes an angle outside the URDF's limit even mid-convergence.
3. Write a small test script (can be a plain Node/ts-node script, doesn't need a full test framework) that runs the solver against all 6 key coordinates from key_config.json plus 2-3 arbitrary points, and prints whether each converged within tolerance and how many iterations it took. Run this and paste me the output.

Commit: `feat: implement damped least squares IK solver with convergence tests`

Checklist before moving on:

- ☐ Solver converges for all 6 key coordinates within the tightened tolerance (1-2mm)

- ☐ No returned joint angle ever exceeds its URDF limit, even for edge-case targets
- ☐ Solver has zero React/Three.js imports (grep to confirm)
- ☐ Test script output has been reviewed and pasted/shown to the team

Prompt 7 — MotionController: the safety gate (Plan mode first, then Build) — *Phase 2 foundation, reused by every later Phase*

[Plan mode] Propose the MotionController design from PROJECT_OVERVIEW.md Section 7. Specifically:

1. Show me the TypeScript types: the MotionCommand union (jog, moveTo, rotateJoint, pressKey — exact shape as defined in PROJECT_OVERVIEW.md Section 6), and the result type (something like `{ accepted: boolean, reason?: string, newAngles?: number[] }`).
2. Describe the resolve step: how each MotionCommand variant gets turned into a target end-effector position (or direct joint delta for rotateJoint, which doesn't need IK at all — it's a direct joint write after limit validation).
3. Describe the validation step: joint limit check (from the URDF, loaded once, not re-typed), workspace/reachability check (did the IK solver converge), and a NaN/sanity check on the returned angles.
4. Describe how a rejected command's reason string is generated (this exact string gets reused by voice feedback in Prompt 11 and the agentic layer in Prompt 14, so make the wording operator-friendly now, not an internal error code — e.g. "Target position is outside joint 3's range" rather than "IK_FAIL_J3").
5. Confirm: this must be the ONLY code path in the entire app allowed to write to the joint-state store from Prompt 3. Every adapter (joystick, keyboard, voice, PIN sequencer, agentic) will call this and nothing else touches joint angles directly.

Show me this plan before writing any code.

Then Build mode:

Implement the MotionController exactly as planned. Additional requirements:

1. Smooth animation: when a command is accepted, don't snap instantly to the new angles – interpolate over a short duration (e.g. 150-300ms) so manual jogging and PIN entry both look physically plausible on camera during the demo.
2. Maintain a rejection/activity log as an array in the store (or a dedicated small store) recording: timestamp, source adapter name (to be passed in as a parameter by whichever adapter calls this), command, accepted/rejected, reason if rejected. Prompt 12 will surface this log in the UI.
3. Add a simple manual test: temporarily wire a single test button that sends a moveTo command to a known reachable point (e.g. key 1's coordinates) and confirm in the telemetry panel that the end-effector actually arrives there within tolerance. Remove or comment out this test button once confirmed (real adapters arrive in Prompts 8-11).

Commit: `feat: implement MotionController as the single validated command gate`

Checklist before moving on:

- ☐ MotionController is confirmed as the only writer to joint state (grep the joint-state store's setter — it should only be called from within this module)
 - ☐ A valid moveTo command to a key coordinate actually moves the arm there, visible in the telemetry panel, within the $\pm 5\text{mm}$ /tightened IK tolerance
 - ☐ An intentionally out-of-range target (e.g. far outside max reach) is correctly rejected with a readable reason string
 - ☐ The activity/rejection log array is populated correctly for both accepted and rejected commands
 - ☐ Animation is smooth, not an instant snap
-

Prompt 8 — GUI joystick control (Build mode) — *Phase 2, task 2*

Build the on-screen joystick control. Steps:

1. Create a 2D drag pad (a square div/canvas, doesn't need to be fancy) representing X/Y jogging: dragging up/down/left/right emits `jog` MotionCommands with deltaX/deltaY proportional to drag distance.
2. Add a separate control for Z (a vertical slider, or a scroll-wheel listener on the drag pad, or a +/- button pair – pick the simplest to implement well) that emits `jog` commands with deltaZ.
3. Throttle emission to roughly 20-30 updates per second while dragging – do not fire a MotionCommand on every single pixel of mouse movement, or the MotionController's IK solve will lag and feel unresponsive.
4. Every emitted command must call the MotionController from Prompt 7, passing "joystick" as the source adapter name for logging purposes.
5. Add this control into the dashboard layout, positioned near the telemetry panel from Prompt 5 (exact placement can be adjusted in Prompt 12/13, don't over-invest in layout polish here).
6. Show a visual "active" state while dragging (e.g. a highlighted border) so it's clear during demo when the joystick is actively being used versus idle.

Commit: `feat: add GUI joystick control wired to MotionController`

Checklist before moving on:

- ☐ Dragging the pad visibly moves the arm smoothly in the 3D view
- ☐ Z control works independently of X/Y
- ☐ Rapid dragging doesn't cause visible lag or command queue buildup
- ☐ Source adapter name "joystick" appears correctly in the activity log from Prompt 7

Prompt 9 — Keyboard control (Build mode) — *Phase 2, task 3*

Add keyboard controls. Requirements:

1. Arrow keys (Up/Down/Left/Right) jog X/Y via the same ``jog`` MotionCommand path as the joystick.
2. Choose and implement a modifier scheme for Z and joint rotation – suggested default: Shift+Up/Down jogs Z, number keys 1-7 select which joint is "active," then +/- (or]/[) rotates the active joint by a fixed increment (e.g. 5 degrees per press) via ``rotateJoint`` commands. You can choose a different scheme if you have a reason, but document whichever you pick.
3. Every keypress must go through the MotionController, passing "keyboard" as the source adapter name.
4. Add a small on-screen legend/help panel showing the current key bindings – this will be pointed at directly during the live demo when explaining "here's manual control via keyboard," so it needs to be readable at a glance, not buried in a tooltip.
5. Make sure keyboard events don't fire when the user is typing in an unrelated text input elsewhere on the page (e.g. the PIN entry field from Prompt 10, once it exists) – scope the listener appropriately or check for focused input elements before handling arrow keys.

Commit: `feat: add keyboard control mapped to the shared MotionCommand pipeline`

Checklist before moving on:

- ☐ Arrow keys move the arm the same way the joystick does (same command path, confirmed via activity log source = "keyboard")
- ☐ Joint selection + rotation modifier scheme works and is documented in the on-screen legend
- ☐ Keyboard controls do NOT fire while typing in another input field on the page
- ☐ Legend is visible without needing to hover/click anything

Prompt 10 — Autonomous PIN entry (Build mode) — *Problem Statement Phase 4 ("Let the Arm Work on Its Own")* — built here out of Phase order, see Evaluation Notes

Implement the autonomous PIN-entry sequencer. Steps:

1. Build a UI: a 6-digit text input (validate: exactly 6 digits, each 1-6, matching the key_config.json key numbers — reject/flag invalid input clearly, e.g. a 7 or a 0 isn't a valid key), a "Run Sequence" button, and a "Stop" button to abort mid-sequence.
2. For each digit in the entered PIN, in order, execute this exact motion pattern via the MotionController (source adapter name: "autonomous"):
 - a. ``pressKey`` step "approach": move to (key.x, key.y, key.z + 0.04) — 40mm above the key.
 - b. ``pressKey`` step "descend": move to (key.x, key.y, key.z) — the actual touch point. This is where the $\pm 5\text{mm}$ tolerance is checked.
 - c. ``pressKey`` step "retract": move back to the approach point before proceeding to the next digit.
3. Add a live step indicator in the UI showing, in real time: which digit index is currently being processed (e.g. "3 of 6"), which sub-phase (approach/descend/retract), and whether the previous digit's descend step was accepted or rejected by the MotionController.
4. If any step is rejected by the MotionController, stop the sequence (don't silently continue to the next digit) and surface the rejection reason clearly in the UI — this matters for judges evaluating correctness, not just happy-path behavior.
5. Confirm every single motion in this sequence, with no exceptions, goes through the MotionController from Prompt 7 — do not write a shortcut that directly sets joint angles for "just this one autonomous case."

Commit: `feat: implement autonomous PIN-entry sequencer with approach/descend/retract motion`

Checklist before moving on:

- ☐ Entering a valid 6-digit PIN (digits 1-6 only) and pressing Run visibly sequences through all 6 keys with a clear approach/descend/retract motion per key
- ☐ Step indicator updates correctly and in sync with the actual arm motion
- ☐ Each descend step lands within the $\pm 5\text{mm}$ tolerance — verify against the telemetry panel's position readout, not just visually
- ☐ Entering an invalid PIN (wrong length, digit 0, digit 7+) is rejected in the UI before any motion is attempted

- ☐ Manually forcing a rejection (e.g. temporarily feeding an unreachable coordinate) stops the sequence and shows the reason, rather than silently continuing

Prompt 11 — Voice control (Build mode) — *Problem Statement Phase 3 ("Talk to the Arm") — required deterministic baseline*

Add voice control using the browser's native Web Speech API

(SpeechRecognition, not any external/paid service). Requirements:

1. A microphone toggle button (start/stop listening), with a clear visual state (listening / idle / error – e.g. if the browser denies mic permission or doesn't support the API, show a readable message rather than failing silently).
2. Parse recognized speech text with a deterministic keyword/regex-based parser (not an LLM – that's Prompt 14's job) supporting at minimum:
 - "move up / down / left / right / forward / back" → `jog` commands
 - "rotate [joint N / base] [left/right] N degrees" → `rotateJoint`
 - "press key N" → `pressKey`
3. Show a live transcript of what was heard, and directly below it, what command (if any) was extracted – including an explicit "command not recognized" state when parsing fails, rather than just doing nothing silently.
4. Every successfully parsed command goes through the MotionController, source adapter name "voice".
5. Build the transcript/feedback display as a reusable component (e.g. VoiceFeedbackPanel) since Prompt 14's agentic bonus will reuse this exact UI pattern for its own conversation log – don't hardcode this panel as voice-control-specific in a way that can't be reused.

Commit: `feat: add voice control via Web Speech API mapped to the shared pipeline`

Checklist before moving on:

- ☐ At least the 3 required command types (jog phrases, rotate phrases, "press key N") are correctly recognized and executed
 - ☐ Transcript displays what was heard, distinct from what was parsed
 - ☐ An unrecognized phrase shows a clear "not recognized" state, not silence
 - ☐ Activity log shows "voice" as the source for these commands
 - ☐ Mic permission denial / unsupported browser shows a readable message instead of a silent failure
-

Prompt 12 — Unify everything into one dashboard (Build mode) — *Integration step, spans Phases 1-4*

Integrate everything built so far into one coherent page layout.

Requirements:

1. Layout: 3D viewer + telemetry (Prompts 3-5) as the dominant/central view. Controls (joystick, keyboard legend, PIN entry, voice) as a tabbed panel or sidebar — your choice, but all five must be reachable without leaving the page or losing the 3D view.
2. Build a single shared "Activity Log" panel that reads directly from the MotionController's log (Prompt 7) — do NOT create separate, duplicated logging per adapter. This log must show, for every command from every source: timestamp, source (joystick/keyboard/voice/autonomous), command summary, accepted/rejected + reason if rejected. This log is your visual proof to judges that all five inputs funnel through one pipeline — it needs to be genuinely fed by one place, not five separate arrays merged for display.
3. Add a "currently active source" indicator (e.g. a small badge showing which control last issued a command) so it's visually obvious during demo which input method is driving the arm at any moment.
4. Smoke-test the full integration: perform a joystick move, a keyboard move, a voice command, and a full PIN-entry sequence, one after another, without reloading the page, and confirm the activity log correctly reflects all of them in order with correct sources.

Commit: `feat: unify all control adapters into one dashboard with shared activity log`

Checklist before moving on:

- ☐ All five control surfaces are reachable from one page without reload
- ☐ Activity log shows entries from all sources tested, each correctly labeled
- ☐ No duplicate/parallel logging arrays exist (grep for where log entries are pushed — should be one place, inside the MotionController)
- ☐ The smoke test in step 4 was actually performed and passed

Prompt 13 — Visual design (this one is for Antigravity, not DeepSeek)

The structure and logic from Prompts 1-12 must already exist and work before starting this — design work on a non-functional page wastes the design pass. Give Antigravity the following design brief, adapted as a single prompt or as a working session brief:

Design brief to hand to Antigravity

This is a browser-based robotic arm simulation and control dashboard for an industrial engineering audience (Vantage Robotics) – NOT a consumer or playful product. All functionality already exists and works; this pass is visual/UX polish only. Do not change any logic, state management, data flow, or the MotionCommand pipeline – component behavior must be unchanged, only appearance and layout.

Visual direction:

- Tone: industrial, precise, trustworthy – think engineering control-room software, not a marketing site. Dark-mode-friendly base (e.g. dark slate/graphite background) with a single confident accent color (amber or orange reads well here, echoing typical robotics/hazard signage – but pick one accent and use it consistently, don't multi-color the UI).
- Typography: a clean sans-serif (system font stack or Inter/similar) – monospace specifically for all numeric telemetry (joint angles, positions) so digits align and don't visually "jitter" as they update live – this matters a lot here since numbers change every frame.
- Hierarchy: the 3D viewer must visually dominate the layout – it's the actual proof of correctness, everything else supports it. Telemetry and controls are important but secondary, styled to be legible at a glance without competing with the 3D view for attention.
- Activity log: use color to distinguish accepted (neutral/success, e.g. a subdued green or the accent color) vs rejected (warning color, e.g. a clear but not alarmist red/amber) entries – this log is one of the things judges will look at directly, so its readability matters.
- Controls: give every interactive control (joystick pad, keyboard legend, PIN input, voice toggle) a clear idle / active / disabled visual state – during a live demo, judges should be able to tell at a glance which control is currently being used.
- Loading/empty states: the 3D viewer's loading state (while URDF parses) and any empty states (e.g. activity log before any command has been issued) should look intentional, not broken or blank.
- Responsiveness: optimize primarily for a laptop/projector display (this will be demoed on a laptop screen or projected) – don't over-invest in mobile responsiveness for a robotics control dashboard.

Do NOT:

- Change any component's props, state, or event handlers
- Remove or hide any of the five control surfaces
- Introduce new dependencies for animation/styling beyond what's already in package.json unless there's a strong specific reason

- Restyle the activity log or telemetry panel in a way that makes numbers harder to scan quickly – legibility beats aesthetics here

Commit (once design pass is complete and re-verified against Prompt 12's checklist): `style: visual design pass on dashboard layout and control components`

Checklist before moving on:

- ☐ Every checklist item from Prompt 12 still passes after the design pass (functionality unchanged)
- ☐ 3D viewer is visually the largest/most prominent element on the page
- ☐ Telemetry numbers use a monospace font and don't visually jump around as they update
- ☐ Accepted vs rejected log entries are distinguishable by color at a glance
- ☐ Active/idle/disabled states are visually clear on every control

Prompt 14 — Agentic voice bonus, Phase 3B (Plan mode first, then Build — optional, only after 1-13 are solid) — *Problem Statement Phase 3B (optional bonus)*

```
[Plan mode] Propose the Phase 3B agentic layer per PROJECT_OVERVIEW.md: a
Next.js API route that takes free-form natural language (typed or
transcribed), sends it to Gemini Flash with function-calling/structured
output constrained to the exact MotionCommand schema, and returns either a
valid command or a clarifying question if the instruction is ambiguous.
Critically: this route must NOT execute anything – it only proposes a
MotionCommand, which the client then runs through the existing
MotionController (Prompt 7) exactly like every other adapter, so an
invalid/out-of-bounds suggestion gets rejected the same way. This is an
ADDITIONAL mode, not a replacement – the deterministic keyword-based voice
control from Prompt 11 must remain fully intact and independently
selectable/demoable after this is built, per the problem statement's
explicit requirement that Phase 3B does not replace the required Phase 3
baseline. Do not modify, remove, or route through Prompt 11's component
when implementing this. Show me the API contract before implementing.
```

Then Build mode:

Implement the Phase 3B route and UI. Requirements:

1. API route (app/api/agentic/route.ts or agreed path): accepts a text instruction, calls Gemini Flash with function-calling constrained to the MotionCommand schema, and returns either a proposed command or a clarifying-question response if the instruction is ambiguous (per the PDF: "ask a clarifying question, instead of guessing").
2. Reuse the VoiceFeedbackPanel pattern from Prompt 11 for a chat-style conversation log: user instruction → what was understood → what happened (accepted/rejected + reason from the MotionController).
3. Every proposed command from this route must be run through the MotionController on the client exactly like Prompts 8-11 – never execute directly from the API route's response.
4. Add a hardcoded fallback: if the Gemini call fails or rate-limits during testing/demo, show a pre-written example exchange (e.g. "nudge the tip toward the panel and press key 3") so the feature can still be demoed even if the live API misbehaves – per DEPLOYMENT.md's guidance on demo-safe fallbacks.
5. Confirm both voice modes (Prompt 11's deterministic keyword parser, and this agentic layer) still work independently and can be demonstrated separately, side by side, without one having broken the other.

Commit: feat: add optional agentic natural-language control layer (Phase 3B bonus)

Checklist before moving on:

- ☐ Prompt 11's deterministic voice control still works exactly as before, unmodified
 - ☐ A free-form, multi-step, or ambiguous instruction gets correctly interpreted OR triggers a clarifying question, not a silent wrong guess
 - ☐ An out-of-bounds command proposed by the agentic layer is correctly rejected by the MotionController, same as any other adapter
 - ☐ The hardcoded fallback exchange works with the live API disabled/blocked (test this deliberately)
-

Prompt 15 — Deploy-readiness pass (Build mode) — Section 8 Deliverables (deployed URL, bonus) — not a numbered Phase

Run a full pre-deploy check:

1. Run ``npm run build`` and confirm it completes with zero errors.
2. Search the codebase for `console.log/print` debug statements in production code paths (not test files) and list them.
3. Confirm every environment variable referenced anywhere in the code (grep for `process.env`) exists in `.env.example`.
4. List any `TODO/FIXME` comments remaining in the codebase.
5. Confirm no `.env` or `.env.local` file is tracked by git (`git status / git log --all --full-history -- .env*`).

Report all findings clearly. Do not auto-fix anything without telling me what you're about to change first.

Commit: `chore: pre-deploy readiness pass`

Checklist before moving on:

- ☐ Build completes cleanly
 - ☐ Debug console.logs reviewed and removed or justified
 - ☐ Every env var is documented in `.env.example`
 - ☐ No secrets in git history
 - ☐ App actually deployed to Vercel and the live URL tested from a different device/network than the dev machine
-

Prompt 16 — Documentation for judges (Build mode) — *Rubric: "System Architecture & Concept Explanation" — not a numbered Phase*

Write README.md and ARCHITECTURE.md.

README.md: setup/run instructions, feature list, tech stack summary, link to the deployed URL.

ARCHITECTURE.md, written for a judge who will read it in about 2 minutes:

1. The "one pipeline, five triggers" framing — explain it in plain language, not jargon.
2. The MotionCommand schema and why every adapter emits the same shape.
3. Why the MotionController is the single validation gate, and what it checks (joint limits, reachability, sanity) before any motion executes.
4. The 7-DOF vs 6-DOF decision (PROJECT_OVERVIEW.md Section 3.1) and which option we picked and why.
5. A simple diagram (Mermaid or ASCII) showing all five adapters (joystick, keyboard, voice, autonomous, agentic-bonus) feeding into the MotionController, and the MotionController feeding the 3D viewer/telemetry/activity log.

Commit: docs: add README and architecture explanation for judging

Checklist before moving on:

- ☐ A teammate who didn't write the code can read ARCHITECTURE.md in under 2 minutes and correctly explain the system verbally afterward
- ☐ The diagram accurately reflects the actual code (not an idealized version that doesn't match what was built)
- ☐ README setup steps work on a clean checkout (test this on a teammate's machine if possible)

Prompt 17 — Electrical schematic writeup (not code — documentation prompt) — *Problem Statement Phase 5 ("Arm's Electrical Schematic")*

Using the pin-mapping reference in PROJECT_OVERVIEW.md Section 9, write ELECTRICAL.md documenting the proof-of-concept wiring reasoning: ESP32 + PCA9685 driver + external servo power rail + common grounding + smoothing capacitor, for powering/controlling this 6-DOF arm's servo motors over Wi-Fi. For each connection, explain WHY it's needed (why servos need external power separate from the MCU, why common ground matters across all components, why a smoothing capacitor is needed near the driver) so the wiring can be reasoned about, not just copied. Do not generate a Wokwi/Tinkercad project file or simulator JSON — just the written reasoning and a labeled connection table, so the actual diagram gets built by hand in Wokwi.

Commit: `docs: add electrical schematic reasoning and pin-mapping writeup`

Checklist before moving on:

- ☐ Every connection in the table has a stated reason, not just a wiring instruction
 - ☐ No Wokwi/Tinkercad project file was generated — only markdown reasoning
 - ☐ The team has actually built the real Wokwi diagram by hand using this writeup as reference
-

System connection map — how everything ties to the dashboard

Control surface	Built in	Emits	Feeds into	Dashboard shows
GUI Joystick	Prompt 8	jog	MotionController	Live joint angles + end-effector position updating in real time
Keyboard	Prompt 9	jog, rotateJoint, t	MotionController	Same telemetry panel, plus on-screen key legend
Voice (deterministic)	Prompt 11	jog, rotateJoint, t, moveTo	MotionController	Live transcript + parsed-command feedback panel
Autonomous PIN sequencer	Prompt 10	pressKey (×6, sequenced)	MotionController	Step indicator (approach/descend/retract) per digit
Agentic NL layer (bonus)	Prompt 14	any MotionCommand, via Gemini function-calling	MotionController (never bypasses it)	Chat-style conversation log with understood/succeeded/failed status
MotionController (the one gate)	Prompt 7	validated joint angles only	3D viewer + telemetry + activity log	Accept/reject status with reason string, for every command from every source above
3D viewer + telemetry	Prompts 3-5	reads joint-state store	—	The arm itself, plus live joint/position readout
Activity log	Prompt 12	reads MotionController's accept/reject stream	—	Full audit trail: which adapter, what command, accepted or rejected + why

The one rule that keeps this whole map true: nothing except the MotionController is allowed to write to the joint-state store. Every adapter only *proposes* a `MotionCommand`. If any future prompt or teammate suggests "just quickly update the joint angle directly for testing," don't — route it through the MotionController anyway, or the activity log and safety validation silently stop being true for that path.